WOXSEN UNIVERSITY | SCHOOL OF TECHNOLOGY | DBMS FINAL REVIEW

Railway Ticket Reservation and Journey Management System

REVIEW 3: Final System Implementation, Verification, and Live Demonstration

Student Name: Heman | Roll Number: 25WU0102097 | Section: AIML Rhinos
Faculty Mentor: Prof. Sahithi | School of Technology, Woxsen University | Week 16

Final System Architecture and Technology Stack

Production Multi-Tier Implementation Flow

Web Frontend

Bootstrap 5 & HTML5
Responsive Navbar & Alerts
ERS Ticket Slips
JavaScript Auto-loader

Application Core

Python 3.9 & Flask
8 Modular Blueprints
Form Validations
Custom Jinja2 Filters

Business Engine

Availability Service [s, e)
Transaction Safe Booking
Pessimistic Row Locks
Atomic WL Promotion

Database Tier

MySQL 8.x InnoDB Engine
14 Relations in 3NF
4 Relational Views
12 SQL Benchmark Reports

Implemented ER Diagram and Database Design

Fully Consistent 14-Table 3NF Relational Schema

✔ **Identical Schema Consistency:** The implemented database schema strictly matches the ER design from Review 1 and normalization proofs from Review 2 without discrepancies.

✔ **Entity Separation:** Stations, Routes, Ordered Stops, Locomotive Definitions, Dated Runs, Rolling Stock Coaches, and Individual Seat Units are completely decoupled.

✔ **Transactional Integrity:** Bookings, Journey_Segments, Waitlists, Payments, and Cancellations maintain complete auditability through foreign keys and composite unique constraints.

✔ **Monetary Precision:** All currency columns use DECIMAL(10,2) eliminating binary floating-point rounding errors.

Completed Application Modules Overview

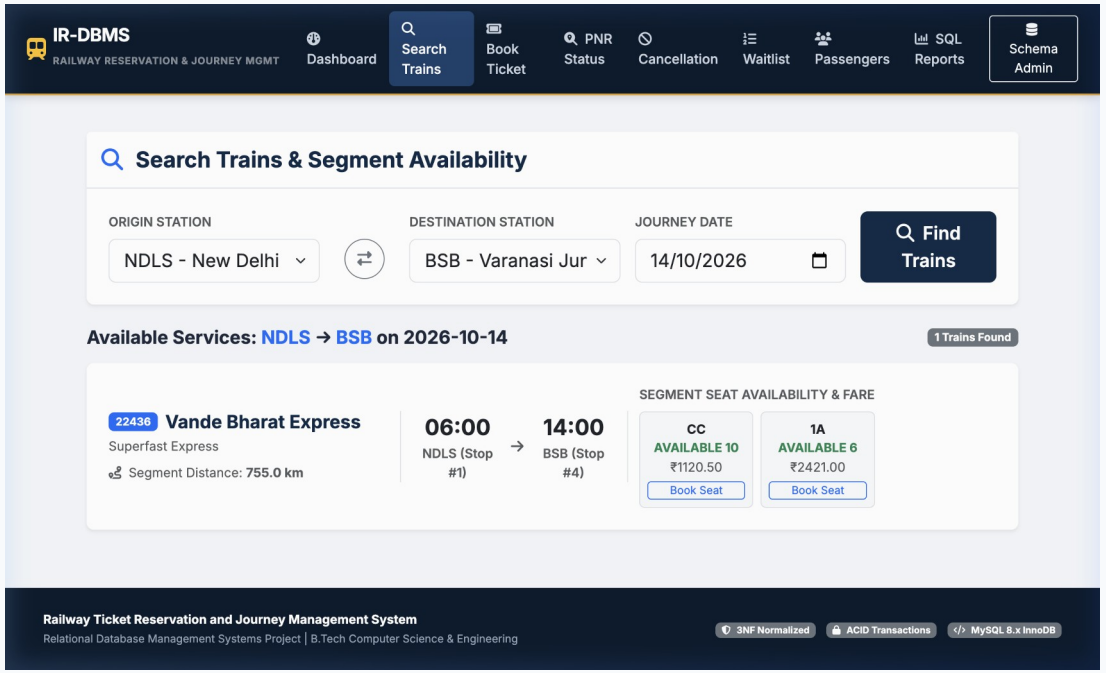
Implemented Software Modules (100% Complete)

- **Dashboard Module (/):** Live database KPI summary cards: total trains, passengers, bookings, waitlists, revenue.
- **Train Search & Timetable (/search):** Origin/destination route matching, stop timetable ordering, live segment seat availability.
- **Passenger Profile Module (/passengers):** Passenger registration with age domain check (1-119) and government ID validation.
- **Reservation & ERS Module (/book, /booking/<pnr>):** Atomic booking, berth allocation, PNR generation, payment simulation, and printable ERS slip.
- **Cancellation & Refund Module (/cancel):** PNR lookup, refund preview (20% fee deduction), and automated waitlist promotion trigger.
- **Waitlist Registry (/waitlist):** Live FIFO queue position tracking across runs and travel classes.
- **SQL Reports Hub (/reports):** Interactive execution of all 12 benchmark SQL reports with dynamic data tables.

Train Search and Live Segment-Wise Availability

Operational Feature Explanation

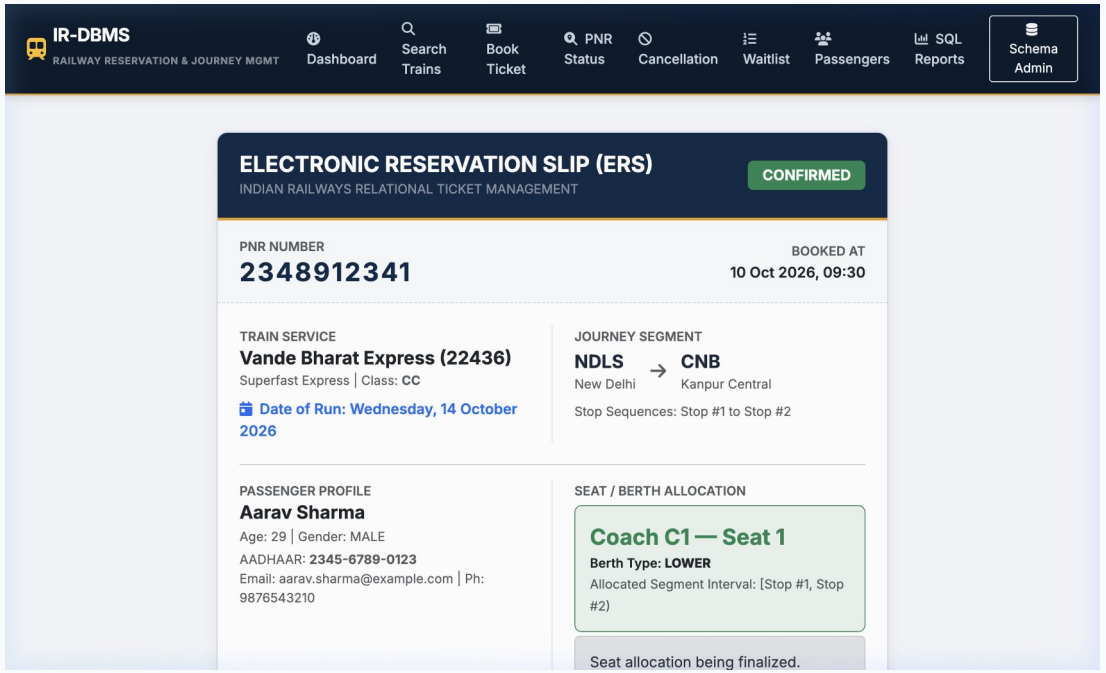
- User selects Origin (NDLS), Destination (BSB), and Travel Date (2026-10-14).
- Relational query validates route progression (Stop 1 precedes Stop 4).
- Segment availability algorithm tests interval overlap across all physical seats in CC and 1A coaches.
- Live status cards display remaining free seats or active waitlist positions with calculated fares.



Confirmed Reservation and Seat Allocation

Electronic Reservation Slip (ERS) Workflow

- Transaction safety: Row-level lock (FOR UPDATE) acquired during seat assignment.
- Generates unique 10-digit PNR and binds passenger profile.
- Inserts active JourneySegment over [1, 2) and records completed Payment.
- Renders printable Electronic Reservation Slip showing Coach, Seat number, and Berth Type.



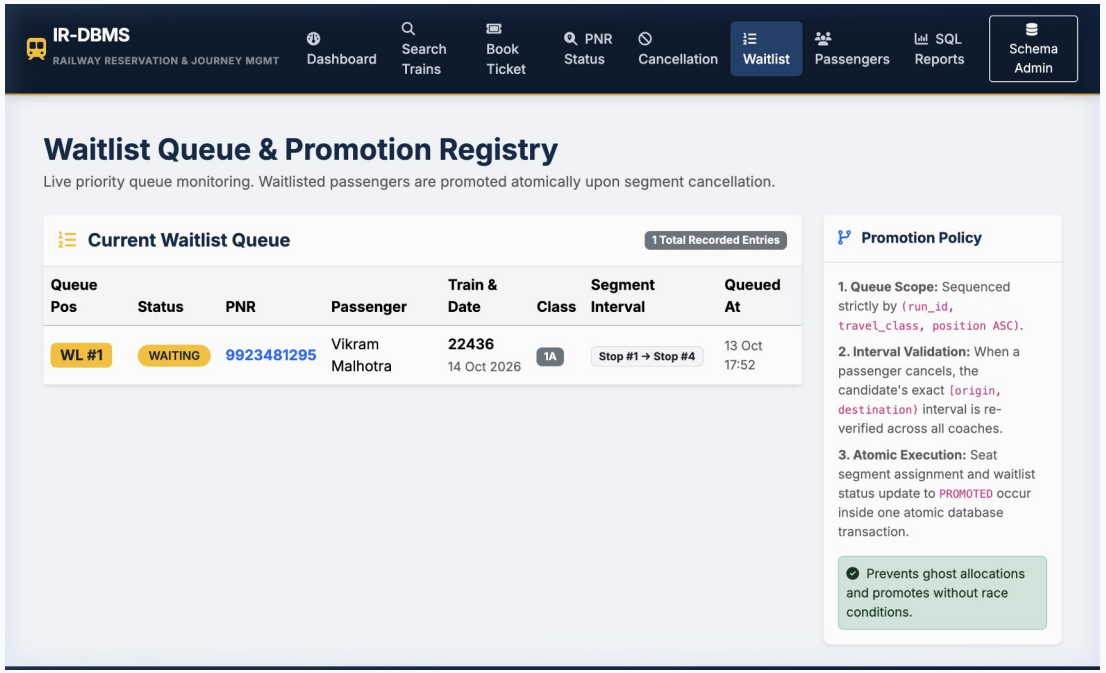
Waitlist Queue and Automated Promotion

Atomic FIFO Promotion Logic

- Exhausted Capacity: When class capacity is full, passenger is queued at Position WL #1.

- Cancellation Trigger: Cancelling a confirmed booking deducts a 20% cancellation fee, refunds 80%, and marks the segment CANCELLED.

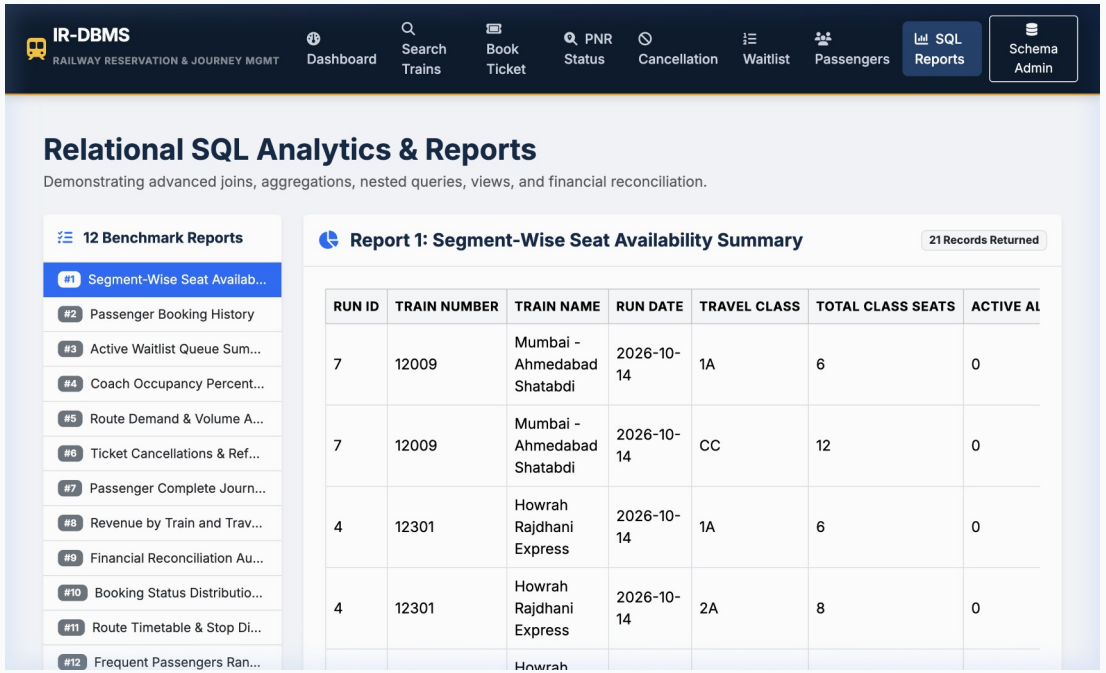
- Atomic Promotion: The system checks the waitlist queue in FIFO order, re-verifies segment availability, assigns the freed seat, and updates status to PROMOTED in one atomic transaction.



SQL Reports Hub and Database-Backed CRUD

12 Real-Time Analytical SQL Queries

- Interactive Reports Hub: Select between 12 benchmark SQL reports.
- Demonstrates Joins, GROUP BY, HAVING, and scalar subqueries.
- Financial Reconciliation: Audits gross booking value vs payments collected vs refunds disbursed.
- Coach Occupancy: Computes exact percentage occupancy per physical carriage.



Transaction Boundaries, Locking, and Integrity

ACID Enforcement and Concurrency Control

✔ **Atomicity:** db.session.begin_nested() ensures that if seat assignment or payment fails, all related insertions roll back cleanly.

✔ **Consistency:** Declarative CHECK constraints (age > 0, fare >= 0, dest > orig) and Foreign Keys maintain valid relational states.

✔ **Isolation & Row Locking:** SELECT … FOR UPDATE places exclusive row locks on TrainRun / Seat rows, serializing competing reservations and eliminating double-booking race conditions.

✔ **Durability:** InnoDB transaction write-ahead logs (redo_log) guarantee data persistence across sudden server crashes.

✔ **Indexing Performance:** B-Tree indexes on (run_id, seat_id, allocation_status) allow interval overlap checks to execute in sub-millisecond time.

Testing Methodology and Verified Results

16 Automated Tests Executed via pytest (100% Pass Rate)

Test Case	Category	Verification Goal	Result
test_interval_overlap_logic	Algorithm	Mathematical half-open interval overlap evaluation	PASSED
test_non_overlapping_seat_reuse	Business Logic	Seat C1-1 concurrently held on [1,2) and [2,4)	PASSED
test_conflicting_interval_detection	Integrity	Conflicting request for [1,4) correctly rejected	PASSED
test_cancelled_allocation_restores_availability	State Machine	Cancelling segment frees seat for subsequent booking	PASSED
test_eligible_cancellation_and_refund	Financial Audit	20% fee deducted; 80% refunded to passenger	PASSED
test_duplicate_cancellation_rejected	Transactions	Second cancellation attempt raises ValueError	PASSED
test_automated_waitlist_promotion	Queue Logic	Freeing confirmed seat promotes WL candidate atomically	PASSED
test_all_12_reports_execute_successfully	SQL Analytics	All 12 benchmark queries execute without errors	PASSED

Limitations and Future Enhancements

Current Prototype Limitations

- Simulated Financial Gateway: Simulated UPI/Card processing without live banking webhooks.
- Static Timetables: Fixed scheduled departures without real-time GPS locomotive tracking.
- Single RDBMS Node: Local relational database instance rather than distributed multi-region cluster.

Future Scope & Production Scaling

- Real-Time NTES Integration: Live train delay tracking and platform assignment feeds.
- Redis Inventory Caching: High-throughput memory bitsets to handle massive concurrent Tatkal spikes.
- Dynamic AI Pricing: Predictive demand modeling adjusting fares and estimating waitlist confirmation probability.
- Read Replicas: Offloading analytical reporting queries to MySQL read replicas.

Conclusion and Viva Voce Preparation

Key Academic Takeaways & Ready Viva Defenses

Q: Why is 3NF necessary?
A: Decoupling train definitions from dated runs, and passenger profiles from bookings, prevents update and insertion anomalies.

Q: How is seat reuse solved?
A: Half-open intervals [s, e) tested via s < r_end AND r_start < e allow disjoint passenger journeys on the same seat.

Q: How are race conditions prevented?
A: ACID transactions with pessimistic row locking (SELECT … FOR UPDATE) serialize competing requests.

Q: Why use DECIMAL?
A: Fixed-point DECIMAL(10,2) prevents binary floating-point rounding errors in fares and refunds.

Q: How are waitlists promoted?
A: Automated FIFO queue evaluation re-checks segment availability and promotes candidates atomically.